



รายงานฉบับสมบูรณ์

เรื่อง

การเปรียบเทียบการสร้าง Java Unit Test

ด้วยอัลกอริทึมและเครื่องมือปัญญาประดิษฐ์ช่วยสร้างชุดทดสอบ

SQA Defects4J Benchmark:

Hill Climbing, Alternating Variable Method (AVM), GPT และ Gemini

จัดทำโดย

กรมภักฎ์ พิริยะ รหัสนักศึกษา 673380262-4

วัชรพล ดวงกองเงิน รหัสนักศึกษา 673380290-9

ศิระพัทธ์ วงศ์วิวัฒน์เสรี รหัสนักศึกษา 673380293-3

เสนอ

ผศ.ดร. ชิตสุธา สุ่มเล็ก

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา

CP353201 Software Quality Assurance

ภาคการศึกษา 1 ปีการศึกษา 2569

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

งานวิจัยนี้เปรียบเทียบการสร้างชุดทดสอบหน่วยภาษา Java โดยอัตโนมัติด้วย Hill Climbing, Alternating Variable Method (AVM), GPT และ Gemini บนชุดข้อมูล Defects4J โดยใช้โปรแกรมรุ่นที่แก้ไขแล้ว (fixed revision) เพื่อสร้างชุดทดสอบและผลลัพธ์คาดหวัง (test oracle) จากนั้นประเมินชุดทดสอบเดียวกันบนโปรแกรมรุ่นที่แก้ไขแล้วและรุ่นที่ยังมีข้อบกพร่อง ตัวชี้วัดประกอบด้วยอัตราชุดทดสอบที่ใช้งานได้ การตรวจพบบั๊ก ความครอบคลุมของโค้ดระดับคำสั่ง ทางเลือก และบรรทัดของคลาสเป้าหมาย จำนวนกรณีทดสอบ JUnit และเวลาที่ใช้

ผลการทดลองจาก 854 เคสต่อวิธีพบว่า GPT ตรวจพบบั๊กได้มากที่สุดจำนวน 38 เคส คิดเป็น 4.45% ตามด้วย Hill Climbing 12 เคส AVM 11 เคส และ Gemini 8 เคส ส่วน Gemini ใช้เวลาเฉลี่ยต่องานทดลองที่ดำเนินการสำเร็จน้อยที่สุด คือ 7.45 วินาที Hill Climbing และ AVM มีอัตราชุดทดสอบที่ใช้งานได้ประมาณ 98% เมื่อพิจารณาเฉพาะงานทดลองที่ดำเนินการสำเร็จ แต่ไม่รองรับ 503 เคสต่อวิธี เมื่อใช้เคสทั้งหมดเป็นตัวหาร ทั้งสองวิธีมีส่วนชุดทดสอบที่ใช้งานได้ 40.28% ขณะที่ GPT และ Gemini มีสัดส่วน 65.69% และ 50.12% ตามลำดับ

ข้อมูลผลการทดลองประกอบด้วยสอง experiment_id และแต่ละวิธีมีข้อจำกัดด้านจำนวนกรณีทดสอบแตกต่างกัน ผลรวมจึงอธิบายได้เฉพาะเครื่องมือและการกำหนดค่าที่ใช้ในการทดลองครั้งนี้ เมื่อพิจารณา 231 เคสที่ทุกวิธีสร้างชุดทดสอบที่ใช้งานได้ GPT มีความครอบคลุมของโค้ดเฉลี่ยสูงที่สุดและตรวจพบบั๊กได้ 10 เคส เทียบกับ Hill Climbing 8 เคส AVM 7 เคส และ Gemini 3 เคส อย่างไรก็ตาม หลักฐานยังไม่เพียงพอที่จะสรุปความเหนือกว่าโดยทั่วไปหรือความแตกต่างอย่างมีนัยสำคัญทางสถิติ ผลการทดลองสะท้อนว่าความครอบคลุมของโค้ดและจำนวนกรณีทดสอบเพียงอย่างเดียวไม่รับประกันการตรวจพบบั๊ก ความถูกต้องของผลลัพธ์คาดหวัง การสร้างออบเจกต์และการเตรียมสถานะ ตลอดจนการเก็บข้อมูลเพื่อให้ตรวจสอบย้อนกลับได้ ล้วนมีผลต่อคุณภาพของการประเมิน

สารบัญ

บทคัดย่อ	ก
สารบัญตาราง	ค
สารบัญรูปภาพ	ง
บทที่ 1 บทนำ	1
1.1 วัตถุประสงค์ของการทดลอง	1
1.2 คำถามการทดลอง	1
1.3 ขอบเขตการทดลอง	1
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 วิธีที่นำมาเปรียบเทียบ	3
2.1 อัลกอริทึมเชิงค้นหา	3
2.1.1 Hill Climbing (HC)	3
2.1.2 Alternating Variable Method (AVM)	3
2.2 เครื่องมือ AI สำหรับสร้างชุดทดสอบ	4
2.2.1 GPT	4
2.2.2 Gemini	4
บทที่ 3 การออกแบบและขั้นตอนการทดลอง	5
3.1 ชุดข้อมูลและการแบ่งงาน	5
3.2 เงื่อนไขการสร้างชุดทดสอบและความเป็นธรรมในการเปรียบเทียบ	5
3.3 การกำหนดค่าและสภาพแวดล้อมการทดลอง	6
3.4 แผนภาพขั้นตอนการประเมิน	7
3.5 ตัวชี้วัดและวิธีวิเคราะห์	8
บทที่ 4 ผลการทดลอง	9
4.1 ความครบถ้วนของข้อมูลและความสามารถในการสร้างชุดทดสอบ	9
4.2 การตรวจพบบั๊กและความครอบคลุมของโค้ดในผลรวม	10
4.3 เวลา จำนวนกรณีทดสอบ และประสิทธิภาพ	11
4.4 ผลบนเคสเดียวกัน	12
4.5 การตรวจพบบั๊กแยกตามโปรเจกต์	13
4.6 ปัญหาที่พบและผลกระทบ	14
4.6.1 ข้อจำกัดของขอบเขตการค้นหาและการสร้างออบเจกต์	14
4.6.2 ปัญหาการคอมไพล์และการตรวจสอบชุดทดสอบจาก AI	14
4.6.3 ข้อผิดพลาดที่ยังคงอยู่ในชุดข้อมูล	15
บทที่ 5 สรุปผลการเรียนรู้และอภิปรายผล	16
5.1 สรุปผล	16
5.2 ข้อจำกัด	17
5.3 ข้อเสนอแนะและแนวทางพัฒนา	18
เอกสารอ้างอิง	19

สารบัญตาราง

ตารางที่ 3.1	การแบ่งช่วงเคสของสมาชิก	5
ตารางที่ 3.2	การกำหนดค่าการทดลอง	6
ตารางที่ 4.1	ความครบถ้วนของผลการทดลอง	9
ตารางที่ 4.2	ผลการตรวจพบบั๊กและความครอบคลุมของโค้ด	10
ตารางที่ 4.3	เวลาและจำนวนกรณีทดสอบ	11
ตารางที่ 4.4	ประสิทธิภาพต่อกรณีทดสอบ	12
ตารางที่ 4.5	ผลการเปรียบเทียบบนเคสเดียวกัน	12
ตารางที่ 4.6	การตรวจพบบั๊กแยกตามโปรเจกต์	13
ตารางที่ 4.7	สาเหตุของเคสที่ไม่รองรับ	14
ตารางที่ 4.8	ผลการคอมไพล์ชุดทดสอบ	14
ตารางที่ 4.9	ข้อผิดพลาดที่ยังคงอยู่ในผลการทดลอง	15

สารบัญรูปภาพ

รูปที่ 3.1 ขั้นตอนการสร้างและประเมินชุดทดสอบ.....	7
---	---

บทที่ 1

บทนำ

1.1 วัตถุประสงค์ของการทดลอง

- 1.1.1 เปรียบเทียบความสามารถของอัลกอริทึมเชิงค้นหาสองวิธี ได้แก่ Hill Climbing และ Alternating Variable Method (AVM) กับเครื่องมือ AI สองชนิด ได้แก่ GPT และ Gemini ในการสร้างชุดทดสอบหน่วยภาษา Java ที่สามารถคอมไพล์และทำงานผ่านบนโปรแกรมรันที่แก้ไขแล้ว รวมถึงตรวจพบบั๊กบนโปรแกรมรันที่ยังมีข้อบกพร่อง
- 1.1.2 วิเคราะห์ความครอบคลุมของโค้ด จำนวนกรณีทดสอบ และเวลาที่ใช้ ตลอดจนเปรียบเทียบแนวทางการสร้างข้อมูลนำเข้า การสร้างออบเจกต์และการเตรียมสถานะ และการสร้างผลลัพธ์คาดหวัง (test oracle) ของแต่ละวิธี
- 1.1.3 สรุปผลการเรียนรู้ ปัญหาที่พบระหว่างการทดลอง และข้อจำกัดที่ควรพิจารณาเมื่อตีความผลลัพธ์
- 1.1.4 จัดเตรียมโค้ดต้นฉบับ ชุดทดสอบที่สร้างขึ้น ผลการทดสอบ แผนภาพการทำงาน พร้อมคำอธิบายการกำหนดค่า และขั้นตอนการทำซ้ำการทดลอง เพื่อให้สามารถตรวจสอบผลได้

1.2 คำถามการทดลอง

- 1.2.1 แต่ละวิธีสามารถสร้างชุดทดสอบที่ใช้งานได้สำเร็จบ่อยเพียงใด
- 1.2.2 แต่ละวิธีตรวจพบบั๊กได้กี่รายการ
- 1.2.3 ชุดทดสอบของแต่ละวิธีครอบคลุมคลาสเป้าหมายได้มากเพียงใด
- 1.2.4 แต่ละวิธีใช้เวลาในการสร้างและประเมินชุดทดสอบเท่าใด
- 1.2.5 เมื่อพิจารณาเฉพาะขอบเขตที่แต่ละวิธีรองรับ ผลลัพธ์และข้อจำกัดของแต่ละวิธีแตกต่างกันอย่างไร

1.3 ขอบเขตการทดลอง

- 1.3.1 ใช้บั๊กจากชุดข้อมูล Defects4J จำนวน 854 เคส โดยแบ่งเคสให้สมาชิกสามคนรับผิดชอบช่วงที่ไม่ซ้ำกัน และให้แต่ละคนรันทั้งสี่วิธีกับทุกเคสในช่วงของตน
- 1.3.2 เปรียบเทียบวิธีสร้างชุดทดสอบทั้งหมดสี่วิธี ได้แก่ Hill Climbing, AVM, gpt-5.6-terra และ gemini-3.5-flash-lite
- 1.3.3 เรียกใช้เครื่องมือ AI ผ่านเกตเวย์ AI ของมหาวิทยาลัยขอนแก่น โดยยืนยันตัวตนด้วย API key และใช้จุดเชื่อมต่อกับชื่อโมเดลตามที่กำหนดไว้ในไฟล์การกำหนดค่า

- 1.3.4 ใช้โปรแกรมรุ่นที่แก้ไขแล้ว (fixed revision) เพื่อวิเคราะห์โค้ดต้นฉบับ สร้างข้อมูลนำเข้า ค้นหา candidate ของชุดทดสอบ และสร้างผลลัพธ์คาดหวัง (test oracle) โดยไม่ใช้รายละเอียดการแก้ไขเป็นข้อมูลโดยตรง
- 1.3.5 ใช้คลาสที่ถูกแก้ไขตามข้อมูลกำกับของ Defects4J เป็นคลาสเป้าหมาย การทดลองนี้จึงเป็นการสร้างชุดทดสอบแบบทราบคลาสเป้าหมาย ไม่ใช่การค้นหาตำแหน่งบั๊กจากทุกคลาสในโปรเจกต์
- 1.3.6 Hill Climbing และ AVM ใช้ค่าตั้งต้นของการสุ่ม (seed) เดียว ส่วน GPT และ Gemini ใช้การรันหนึ่งครั้งต่อวิธีต่อเคส ผลลัพธ์จึงเป็นผลเชิงพรรณนาของระบบ พรอมต์ และการกำหนดค่าที่ใช้ในการทดลองครั้งนี้
- 1.3.7 วัดความครอบคลุมของโค้ดระดับคำสั่ง ทางเลือก และบรรทัดของคลาสเป้าหมายบนโปรแกรมรุ่นที่แก้ไขแล้ว
- 1.3.8 เวลาที่รายงานเป็นเวลาสำหรับการสร้างและประเมินชุดทดสอบตามข้อมูลที่ระบบบันทึก โดยไม่รวมเวลานำซอร์สโค้ดออกจากระบบควบคุมรุ่นและคอมไพล์โปรเจกต์ในขั้นเตรียมการ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ทำให้ทราบความสามารถของ Hill Climbing, AVM, GPT และ Gemini ในการสร้างชุดทดสอบที่ใช้งานได้และตรวจพบบั๊กภายใต้เงื่อนไขที่กำหนดไว้ในการทดลอง
- 1.4.2 ทำให้เห็นความแตกต่างระหว่างความสามารถในการสร้างชุดทดสอบ ความครอบคลุมของโค้ด และความสามารถในการตรวจพบบั๊ก
- 1.4.3 ทำให้ทราบผลกระทบของการสร้างออบเจกต์ การเตรียมสถานะ ขอบเขตข้อมูลนำเข้า และผลลัพธ์คาดหวัง (test oracle) ต่อคุณภาพของชุดทดสอบที่สร้างขึ้น
- 1.4.4 ทำให้ทราบข้อดีและข้อจำกัดของอัลกอริทึมเชิงค้นหาและเครื่องมือ AI ช่วยสร้างชุดทดสอบ เพื่อใช้เป็นแนวทางในการพัฒนาเครื่องมือสร้างชุดทดสอบในอนาคต
- 1.4.5 ได้โค้ด ชุดทดสอบ ผลการทดลอง และขั้นตอนการทำงานที่สามารถนำไปตรวจสอบวิเคราะห์เพิ่มเติม หรือใช้เป็นพื้นฐานสำหรับการทดลองครั้งต่อไปได้

บทที่ 2

วิธีที่นำมาเปรียบเทียบ

2.1 อัลกอริทึมเชิงค้นหา

Hill Climbing (HC) และ Alternating Variable Method (AVM) ใช้คะแนนความเหมาะสม (fitness) เพื่อประเมิน candidate ของชุดทดสอบ โดยคำนวณจาก $\text{branch_ratio} + 0.001 \times \text{instruction_ratio}$ ของคลาสเป้าหมายบนโปรแกรมรุ่นที่แก้ไขแล้ว สูตรนี้ให้น้ำหนักกับความครอบคลุมระดับทางเลือกเป็นหลัก และใช้ความครอบคลุมระดับคำสั่งเป็นองค์ประกอบรองในการแยกคะแนน หลังการค้นหา ระบบเก็บ candidate ได้สูงสุด 3 รายการต่อเมธอดเป้าหมาย โดยเลือก candidate ที่ให้พฤติกรรมบนโปรแกรมรุ่นที่แก้ไขแล้วแตกต่างกันก่อน เพื่อลดความซ้ำของชุดทดสอบ แนวคิดการใช้การค้นหาเพื่อสร้างข้อมูลทดสอบอธิบายไว้ในงานสำรวจของ McMin [4]

2.1.1 Hill Climbing (HC)

Hill Climbing ค้นหาข้อมูลนำเข้าและการเตรียมสถานะจาก candidate ปัจจุบัน โดยปรับค่าทีละขั้นแล้ววัดคะแนนความเหมาะสม หากคะแนนดีขึ้นจะใช้ candidate ใหม่ค้นหาต่อ หากค้นหาระยะหนึ่งแล้วคะแนนยังไม่ดีขึ้น อาจเริ่มค้นหาใหม่ตามจำนวนครั้งที่กำหนด ระบบคัดเลือก candidate ที่มีพฤติกรรมหลากหลาย แล้วนำข้อมูลนำเข้าที่ได้ไปรันบนโปรแกรมรุ่นที่แก้ไขแล้วเพื่อสร้างคำสั่งตรวจสอบผล (assertion) จุดแข็งคือสามารถค้นหาค่าที่เพิ่มคะแนนได้โดยอัตโนมัติ และตรวจย้อนกลับข้อมูลนำเข้า การเตรียมสถานะ คะแนน และพฤติกรรมได้ อย่างไรก็ตาม การค้นหาอาจหยุดอยู่ที่คะแนนเดิมเมื่อไม่มีข้อมูลนำเข้าใกล้เคียงที่ทำให้คะแนนเพิ่มขึ้น อีกทั้งระบบยังรองรับชนิดข้อมูลนำเข้าและการสร้างออบเจกต์ได้จำกัด

2.1.2 Alternating Variable Method (AVM)

AVM ค้นหาข้อมูลนำเข้าโดยปรับค่าทีละตัวแปร เช่น ลองเพิ่มหรือลดค่าตัวเลขเป็นลำดับ พร้อมสำรวจการเตรียมสถานะที่กำหนดไว้ candidate แต่ละรายการจะถูกรันบนโปรแกรมรุ่นที่แก้ไขแล้วเพื่อวัดคะแนนความเหมาะสมและกำหนดทิศทางการค้นหา หลังการค้นหา ระบบคัดเลือก candidate ที่มีพฤติกรรมหลากหลายได้สูงสุด 3 รายการต่อ 1 เมธอด แล้วนำผลจากโปรแกรมรุ่นที่แก้ไขแล้วมาสร้างคำสั่งตรวจสอบผล จุดแข็งคือมีลำดับการปรับตัวแปรที่อธิบายและตรวจย้อนกลับได้ ข้อจำกัดคืออาจค้นหาได้ยากเมื่อหลายตัวแปรต้องเปลี่ยนพร้อมกัน หรือค่าที่ทดลองให้คะแนนความครอบคลุมของโค้ดเท่ากัน ระบบยังรองรับชนิดข้อมูลนำเข้าและการสร้างออบเจกต์ได้จำกัดเช่นเดียวกับ Hill Climbing หลักการค้นหาโดยสลับปรับตัวแปรอธิบายไว้ในงานของ McMin และ Kapfhammer [5]

2.2 เครื่องมือ AI สำหรับสร้างชุดทดสอบ

GPT และ Gemini ถูกเรียกผ่านเกตเวย์ของมหาวิทยาลัยขอนแก่น โดยข้อมูลการกำหนดค่าระบุโมเดล gpt-5.6-terra และ gemini-3.5-flash-lite และกำหนด temperature เป็น 0 ชื่อโมเดลเหล่านี้เป็นค่าที่เกตเวย์รายงาน โดยไม่มีหลักฐานยืนยันรุ่นภายในของโมเดล GPT และ Gemini ใช้พารามิเตอร์ที่ออกแบบแยกสำหรับแต่ละโมเดล โดยใช้ข้อมูลจาก fixed revision และการกำหนดค่าที่ระบุไว้ในการทดลอง การเปรียบเทียบนี้จึงเป็นการเปรียบเทียบเครื่องมือร่วมกับพารามิเตอร์และการกำหนดค่าที่ใช้จริง ไม่ใช่การควบคุมให้แตกต่างกันเฉพาะตัวโมเดล

ทั้งสองเครื่องมือรับโค้ดต้นฉบับและข้อมูลส่วนต่อประสานสาธารณะ (public API) ของโปรแกรมรุ่นที่แก้ไขแล้วผ่านพารามิเตอร์ แล้วเลือกข้อมูลนำเข้า เตรียมสถานะ และเขียนชุดทดสอบหน่วยภาษา Java พร้อมคำสั่งตรวจสอบผลที่คาดหวัง ระบบนำชุดทดสอบที่ได้ไปคอมไพล์และรันบนโปรแกรมรุ่นที่แก้ไขแล้วก่อน หากผ่านจึงนำชุดทดสอบเดิมไปรันบนโปรแกรมรุ่นที่ยังมีข้อบกพร่องเพื่อตรวจพบบั๊ก ชุดทดสอบใช้รูปแบบของ JUnit 4 [3]

2.2.1 GPT

GPT ใช้ชื่อโมเดล gpt-5.6-terra ตามข้อมูลที่เกตเวย์รายงาน และใช้พารามิเตอร์ที่ออกแบบสำหรับ GPT เพื่อสร้างชุดทดสอบจากข้อมูลของโปรแกรมรุ่นที่แก้ไขแล้ว จุดแข็งของแนวทาง AI คือสามารถสร้างชุดทดสอบที่ใช้ข้อมูลนำเข้าหลายค่า คำสั่งตรวจสอบผลหลายรายการ และกรณีขอบเขตได้ ข้อจำกัดคืออาจเรียก API ผิด เตรียมสถานะที่ใช้งานไม่ได้ หรือคาดหวังผลผิด จนชุดทดสอบคอมไพล์ไม่ผ่านหรือรันบนโปรแกรมรุ่นที่แก้ไขแล้วไม่ผ่าน ผลลัพธ์จึงขึ้นอยู่กับข้อมูลในพารามิเตอร์และการกำหนดค่าที่ใช้ด้วย

2.2.2 Gemini

Gemini ใช้ชื่อโมเดล gemini-3.5-flash-lite ตามข้อมูลที่เกตเวย์รายงาน และใช้พารามิเตอร์ที่ออกแบบสำหรับ Gemini โดยใช้ข้อมูลของโปรแกรมรุ่นที่แก้ไขแล้วเช่นเดียวกัน ชุดทดสอบที่สร้างขึ้นต้องผ่านการคอมไพล์และตรวจสอบด้วยกระบวนการเดียวกับ GPT ก่อนประเมินการตรวจพบบั๊ก ข้อจำกัดของแนวทาง AI ด้านการเรียก API การเตรียมสถานะ และผลลัพธ์คาดหวังยังต้องพิจารณาเช่นเดียวกัน และไม่สามารถระบุถึงความแตกต่างของผลเกิดจากตัวโมเดลเพียงปัจจัยเดียว

บทที่ 3

การออกแบบและขั้นตอนการทดลอง

3.1 ชุดข้อมูลและการแบ่งงาน

ใช้เคสที่เปิดใช้งานใน cases.csv จำนวน 854 เคส เรียงตาม (project, bug_id) แล้วแบ่งเป็นช่วงที่ไม่ซ้ำกัน สมาชิกแต่ละคนรันทั้งสี่วิธีสำหรับทุกเคสของตน ชุดข้อมูลและขั้นตอนการจัดการโปรแกรมแต่ละรุ่นอ้างอิงโครงสร้างของ Defects4J [1]

ตารางที่ 3.1 การแบ่งช่วงเคสของสมาชิก

Member No.	Case Range	Cases	Results	experiment_id
1	1–285	285	1,140	9f661b38cc1eee98
2	286–570	285	1,140	9f661b38cc1eee98
3	571–854	284	1,136	1b010d74235e8785

ผลการทดลองของสมาชิกทั้งสามบันทึก Git commit เดียวกัน อย่างไรก็ตาม สมาชิกคนที่ 3 มี experiment_id แตกต่างจากสมาชิกอีกสองคน เนื่องจากการตั้งค่า End of line sequence เป็น CRLF ในขณะที่สมาชิกคนอื่นใช้ LF ทำให้แม้ว่าจะเป็น config และ commit เดียวกัน แต่ experiment_id จึงไม่เหมือนกัน

3.2 เงื่อนไขการสร้างชุดทดสอบและความเป็นธรรมในการเปรียบเทียบ

ระบบใช้ข้อมูลจากโปรแกรมรุ่นที่แก้ไขแล้วเพื่อสร้างชุดทดสอบ โดยไม่ส่งซอร์สโค้ดของโปรแกรมรุ่นที่ยังมีข้อบกพร่อง รายละเอียดการแก้ไข ความแตกต่างของโค้ด รายงานปัญหา หรือชุดทดสอบที่ใช้กระตุ้นบั๊กให้เครื่องมือสร้างชุดทดสอบ อย่างไรก็ตาม ระบบเลือกคลาสเป้าหมายจากข้อมูล classes.modified ของ Defects4J ซึ่งระบุว่าบั๊กเกี่ยวข้องกับคลาสใด ผลการทดลองนี้จึงเป็นการสร้างชุดทดสอบแบบทราบคลาสเป้าหมาย ไม่ใช่การค้นหาบั๊กโดยไม่ทราบตำแหน่งล่วงหน้า ข้อมูลนี้บันทึกไว้ใน 3,415 งานทดลอง ส่วนอีก 1 งานทดลองล้มเหลวระหว่างเตรียมข้อมูลจึงไม่มีค่านี้

HC และ AVM ค้นหาข้อมูลนำเข้าสำหรับเมธอดสาธารณะที่รับอาร์กิวเมนต์ชนิดพื้นฐาน (primitive) หรือ String ตามที่ระบบรองรับ การสร้างออบเจกต์และการเตรียมสถานะใช้ public API ภายใต้อำนาจด้านความลึก จำนวนชนิด และเวลา ระบบใช้ค่าที่กำหนดไว้สำหรับตัวสร้างออบเจกต์ (constructor) แทนการค้นหาวิธีสร้างออบเจกต์ที่ซับซ้อนอย่างอิสระ

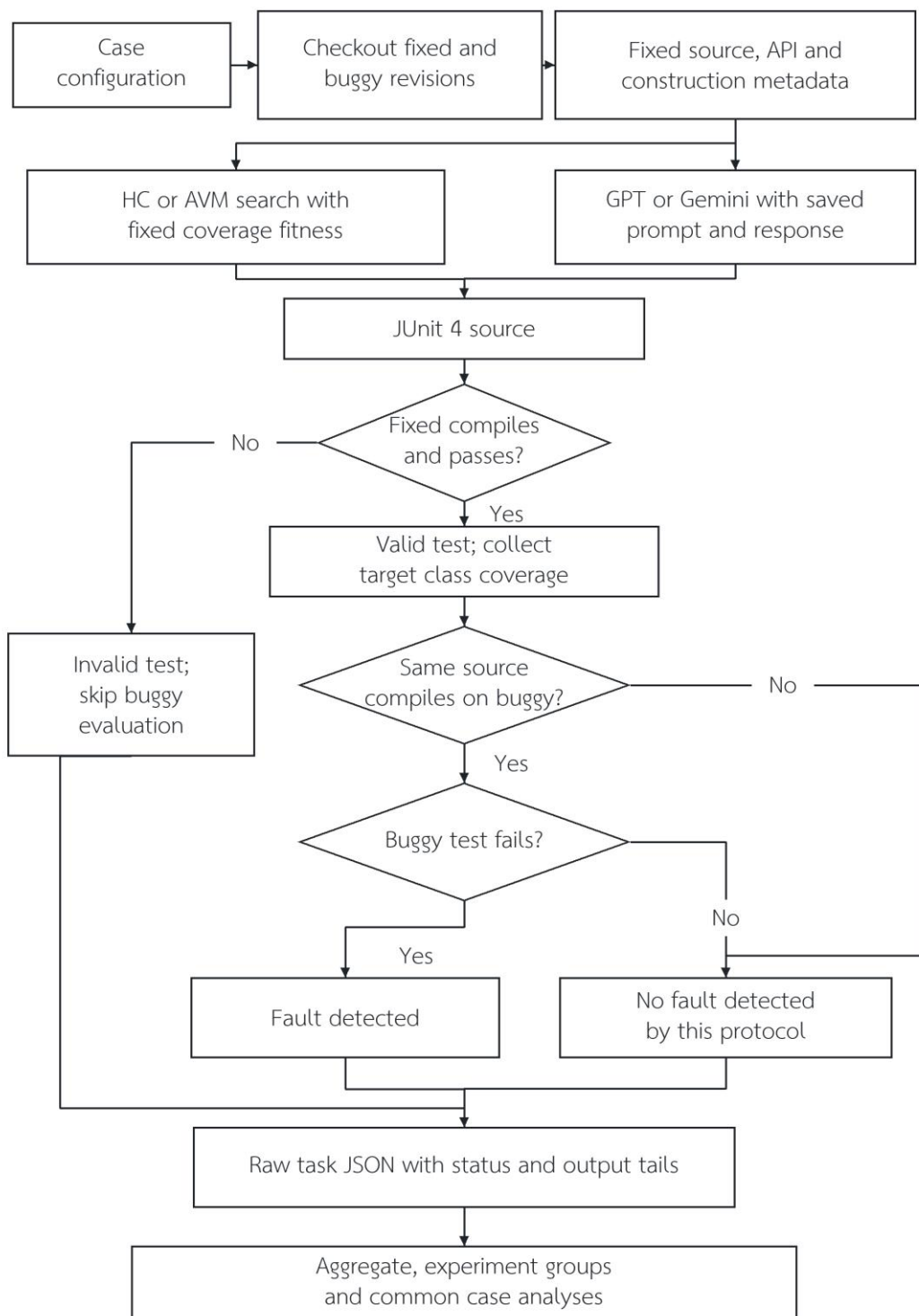
ระบบเตรียมเมธอดเป้าหมายจากข้อมูลกำกับได้สูงสุด 5 เมธอดต่อเคส HC และ AVM สร้างได้สูงสุด 3 กรณีทดสอบต่อเมธอด หรือ 15 กรณีทดสอบต่อเคส ส่วนพารามิเตอร์ของ AI ขอให้สร้าง 1 กรณีทดสอบต่อเมธอด แต่ระบบไม่ได้บังคับให้ AI ทำตามจำนวนนี้หรือเรียกเฉพาะเมธอดในรายการ AI ยังสร้างชุดทดสอบได้เมื่อรายการเมธอดเป้าหมายว่าง ดังนั้นจำนวน กรณีทดสอบ ข้อมูลนำเข้า และคำสั่งตรวจสอบผลที่แต่ละวิธีสร้างจึงไม่เท่ากัน และต้องนำความแตกต่างนี้ไปประกอบการตีความความครอบคลุมของโค้ดและการตรวจพบบั๊ก

3.3 การกำหนดค่าและสภาพแวดล้อมการทดลอง

ตารางที่ 3.2 การกำหนดค่าการทดลอง

Parameter	Source Code / Configuration
การทำซ้ำ HC และ AVM	ใช้ seed 101 และรันหนึ่งครั้ง
การทำซ้ำ GPT และ Gemini	รันหนึ่งครั้งต่อวิธีต่อเคส
เมธอดเป้าหมายและการค้นหา	เมธอดเป้าหมาย 5 เมธอด; ประเมิน 20 ครั้งต่อเมธอด; candidate 3 รายการต่อเมธอด; HC เริ่มค้นหาใหม่ 2 ครั้ง
งบประมาณเวลา	180 วินาทีหลังเตรียมการ; ค้นหา 110 วินาที; สำรองสำหรับขั้นสุดท้าย 60 วินาที
เวลาสูงสุดของ candidate และชุดทดสอบ	30 วินาที / 45 วินาที
การสร้างออบเจกต์	ความลึก 3 ระดับ; ชนิดข้อมูล 24 ชนิด; เวลาสูงสุด 15 วินาที และค้นหาชนิดย่อยแบบจำกัด
การเตรียมสถานะ	การกระทำ 4 รายการ; พารามิเตอร์ 2 ตัว; ขั้นตอน 2 ชั้น; ลำดับการกระทำ 8 ลำดับ
ข้อจำกัดของ AI	โค้ดต้นฉบับ 14,000 อักขระ; ผลตอบกลับ 4,096 โทเคน; เวลาสูงสุดของ API 120 วินาที; ลองใหม่ 2 ครั้ง
สภาพแวดล้อมคอนเทนเนอร์	Ubuntu 20.04, Java 11, เขตเวลา America/Los_Angeles, โคลด C.UTF-8
เครื่องมือวัดความครอบคลุม	JaCoCo 0.8.13 ตาม URL ที่กำหนดไว้ในตัวประเมิน

3.4 แผนภาพขั้นตอนการประเมิน



รูปที่ 3.1 ขั้นตอนการสร้างและประเมินชุดทดสอบ

`valid_test` เป็นคุณสมบัติระดับชุดทดสอบ การตรวจพบบั๊กนับเมื่อชุดทดสอบผ่านบนโปรแกรมรุ่นที่แก้ไขแล้ว คอมไพล์ผ่านบนโปรแกรมรุ่นที่ยังมีข้อบกพร่อง และกระบวนการรันชุดทดสอบบนรุ่นที่ยังมีข้อบกพร่องไม่ผ่าน การดำเนินการสำเร็จไม่ได้หมายความว่าชุดทดสอบใช้งานได้เสมอไป และงานทดลองที่มีชุดทดสอบใช้งานได้แต่เก็บข้อมูลความครอบคลุมของโค้ดไม่สำเร็จจะมีสถานะเป็นข้อผิดพลาด

3.5 ตัวชี้วัดและวิธีวิเคราะห์

อัตราชุดทดสอบที่ใช้งานได้เมื่อดำเนินการสำเร็จ: จำนวนงานทดลองที่ดำเนินการสำเร็จและมีชุดทดสอบใช้งานได้ / จำนวนงานทดลองที่ดำเนินการสำเร็จ

สัดส่วนชุดทดสอบที่ใช้งานได้จากเคสทั้งหมด: จำนวนงานทดลองที่ดำเนินการสำเร็จและมีชุดทดสอบใช้งานได้ / งานทดลองที่กำหนดไว้ทั้งหมด 854 งานต่อวิธี

การตรวจพบบั๊ก: จำนวนคู่ (`project`, `bug_id`) ที่ไม่ซ้ำกันและมีการรันที่ดำเนินการสำเร็จและตรวจพบบั๊ก / 854 เคส

ความครอบคลุมของโค้ด: ค่าเฉลี่ยสัดส่วนความครอบคลุมของงานทดลองที่ดำเนินการสำเร็จมีชุดทดสอบใช้งานได้ และมีข้อมูล โดยวัดเฉพาะคลาสเป้าหมายบนโปรแกรมรุ่นที่แก้ไขแล้วตามตัวชี้วัดของ JaCoCo [2]

เวลา: ค่าเฉลี่ยและมัธยฐานของงานทดลองที่ดำเนินการสำเร็จและมีข้อมูลในฟิลด์ที่เกี่ยวข้อง โดยไม่รวมเวลานำซอร์สโค้ดออกจากระบบควบคุมรุ่นและคอมไพล์โปรเจกต์ในขั้นเตรียมการ เวลาสร้างชุดทดสอบของอัลกอริทึมเชิงคันทารวมเวลาประเมิน `candidate` และการวัดด้วย JaCoCo ระหว่างการค้นหา

ประสิทธิภาพต่อการทดสอบ: นำสัดส่วนความครอบคลุมของโค้ดหรือเวลาหารด้วยจำนวนกรณีทดสอบ ของแต่ละงานทดลอง แล้วจึงคำนวณค่าเฉลี่ย ตัวชี้วัดนี้ใช้ประกอบการวิเคราะห์ และไม่ได้หมายถึงความครอบคลุมที่เพิ่มขึ้นจากการเพิ่มกรณีทดสอบแต่ละกรณี

ค่าที่ไม่มีข้อมูลไม่ถูกตีความเป็นความครอบคลุมของโค้ดหรือเวลาเท่ากับ 0 การวิเคราะห์เพิ่มเติมใช้เคสที่ทุกวิธีดำเนินการสำเร็จจำนวน 349 เคส และเคสที่ทุกวิธีดำเนินการสำเร็จพร้อมมีชุดทดสอบใช้งานได้จำนวน 231 เคส รวมถึงแยกผลตาม `experiment_id` เพื่อแสดงขอบเขตการคัดเลือกและที่มาของข้อมูล

บทที่ 4

ผลการทดลอง

4.1 ความครบถ้วนของข้อมูลและความสามารถในการสร้างชุดทดสอบ

ตารางที่ 4.1 ความครบถ้วนของผลการทดลอง

Method	Tasks	Completed	Unsupported	Errors	Valid Suites	Validity (Valid / Completed) (%)	Valid Yield (Valid / 854) (%)
Hill Climbing	854	349	503	2	344	98.57%	40.28%
AVM	854	350	503	1	344	98.29%	40.28%
GPT	854	849	0	5	561	66.08%	65.69%
Gemini	854	853	0	1	428	50.18%	50.12%

การทดลองประกอบด้วยบักจาก Defects4J จำนวน 854 เคส และใช้วิธีสร้างชุดทดสอบทั้งหมดสี่วิธี จึงคาดว่าจะมีงานทดลองทั้งหมด 3,416 งาน จากการตรวจสอบพบว่าผลลัพธ์ครบตามจำนวนที่กำหนด โดยไม่มีงานทดลองขาด เกิน หรือซ้ำกัน

ผลรวมแบ่งเป็นงานทดลองที่ดำเนินการสำเร็จจำนวน 2,401 งาน ไม่รองรับจำนวน 1,006 งาน และมีข้อผิดพลาดจำนวน 9 งาน โดยไม่พบงานที่ค้างอยู่ในสถานะ paused quota ในชุดข้อมูลสุดท้าย

Hill Climbing ดำเนินการสำเร็จ 349 งาน และสร้างชุดทดสอบที่ใช้งานได้ 344 งาน ส่วน AVM ดำเนินการสำเร็จ 350 งาน และสร้างชุดทดสอบที่ใช้งานได้ 344 งาน ทั้งสองวิธีไม่รองรับ 503 งานเท่ากัน เนื่องจากใช้กลไกวางแผนการสร้างออบเจกต์และขอบเขตเมธอดเป้าหมายร่วมกัน

เมื่อพิจารณาเฉพาะงานทดลองที่ดำเนินการสำเร็จ Hill Climbing และ AVM มีอัตราชุดทดสอบที่ใช้งานได้ร้อยละ 98.57 และ 98.29 ตามลำดับ แสดงให้เห็นว่าเมื่อระบบสร้าง candidate และดำเนินการจนเสร็จ ชุดทดสอบส่วนใหญ่จะคอมไพล์และทำงานผ่านบนโปรแกรมรุ่นที่แก้ไขแล้ว อย่างไรก็ตาม เมื่อหารจำนวนชุดทดสอบที่ใช้งานได้ด้วยเคสทั้งหมด 854 เคส ทั้งสองวิธีมีสัดส่วนร้อยละ 40.28 เนื่องจากมีงานทดลองที่ไม่รองรับจำนวนมาก

GPT ดำเนินการสำเร็จ 849 งาน และสร้างชุดทดสอบที่ใช้งานได้ 561 งาน คิดเป็นอัตราชุดทดสอบที่ใช้งานได้ในงานที่ดำเนินการสำเร็จร้อยละ 66.08 และสัดส่วนจากเคสทั้งหมดร้อยละ 65.69 ซึ่งเป็นสัดส่วนสูงที่สุดในการทดลองนี้ ส่วน Gemini ดำเนินการสำเร็จ 853 งาน และสร้างชุดทดสอบที่ใช้งานได้ 428 งาน คิดเป็นอัตราชุดทดสอบที่ใช้งานได้ในงานที่ดำเนินการสำเร็จร้อยละ 50.18 และสัดส่วนจากเคสทั้งหมดร้อยละ 50.12

ผลดังกล่าวแสดงว่าควรแยกความถูกต้องของชุดทดสอบเมื่อระบบดำเนินการสำเร็จออกจากความสามารถในการรองรับเคสทั้งหมด Hill Climbing และ AVM มีอัตราชุดทดสอบที่ใช้งานได้สูงเมื่อดำเนินการสำเร็จ แต่รองรับเคสได้น้อยกว่า ขณะที่ GPT และ Gemini สร้างโค้ดชุดทดสอบได้ครอบคลุมเคสกว้างกว่า แต่ยังมีงานทดลองที่ดำเนินการสำเร็จบางส่วนซึ่งชุดทดสอบไม่ผ่านเกณฑ์การใช้งาน

4.2 การตรวจพบบั๊กและความครอบคลุมของโค้ดในผลรวม

ตารางที่ 4.2 ผลการตรวจพบบั๊กและความครอบคลุมของโค้ด

Method	Detected Bugs	Detection Rate (/ 854) (%)	Mean Instruction Coverage (%)	Mean Branch Coverage (%)	Mean Line Coverage (%)	Suites for Coverage
Hill Climbing	12	1.41%	21.85%	11.03%	23.25%	344
AVM	11	1.29%	22.34%	11.45%	23.74%	344
GPT	38	4.45%	27.39%	16.02%	27.95%	561
Gemini	8	0.94%	26.55%	15.17%	27.55%	428

จากเคสทั้งหมด 854 เคส Hill Climbing ตรวจพบบั๊กได้ 12 เคส AVM ตรวจพบ 11 เคส GPT ตรวจพบ 38 เคส และ Gemini ตรวจพบ 8 เคส คิดเป็นอัตราการตรวจพบต่อเคสทั้งหมดร้อยละ 1.41, 1.29, 4.45 และ 0.94 ตามลำดับ

GPT ตรวจพบบั๊กได้มากที่สุดในการทดลองนี้ อย่างไรก็ตาม จำนวนดังกล่าวขึ้นอยู่กับทั้งคุณภาพของชุดทดสอบและความสามารถในการสร้างชุดทดสอบที่ใช้งานได้บนเคสที่ Hill Climbing และ AVM ไม่รองรับ จึงไม่ควรตีความผลรวมว่าเป็นความแตกต่างของความสามารถในการค้นหาเพียงอย่างเดียว

เมื่อรวมผลจากทั้งสี่วิธี พบว่าสามารถตรวจพบบั๊กที่ไม่ซ้ำกันได้ทั้งหมด 51 จาก 854 เคส คิดเป็นร้อยละ 5.97 โดย Hill Climbing และ AVM ตรวจพบบั๊กร่วมกัน 11 เคส และ Hill Climbing ตรวจพบ Time-9 เพิ่มเติมอีกหนึ่งเคส นอกจากนี้ มีบั๊ก 32 เคสที่ตรวจพบโดย GPT เพียงวิธีเดียว และมีบั๊ก 5 เคสที่ตรวจพบโดย Gemini เพียงวิธีเดียว ได้แก่ Gson-4, JacksonDatabind-105, Math-31, Math-54 และ Math-95

ค่าเฉลี่ยความครอบคลุมของโค้ดของแต่ละวิธีคำนวณจากงานทดลองที่มีชุดทดสอบใช้งานได้คนละกลุ่มและมีจำนวนตัวอย่างแตกต่างกัน จึงยังไม่ใช้การเปรียบเทียบบนเคสเดียวกันโดยตรง นอกจากนี้ AVM มีความครอบคลุมของโค้ดเฉลี่ยสูงกว่า Hill Climbing เล็กน้อย แต่ Hill Climbing ตรวจพบบั๊กได้มากกว่าหนึ่งเคส แสดงว่าการเพิ่มความครอบคลุมของโค้ดเพียงอย่างเดียวไม่รับประกัน

ว่าจะตรวจพบบั๊กได้มากขึ้น ชุดทดสอบยังต้องมีข้อมูลนำเข้าและสถานะที่กระตุ้นบั๊ก รวมถึงผลลัพธ์คาดหวัง (test oracle) ที่สามารถสังเกตความแตกต่างของพฤติกรรมได้

4.3 เวลา จำนวนกรณีทดสอบ และประสิทธิภาพ

ตารางที่ 4.3 เวลาและจำนวนกรณีทดสอบ

Method	Generated Test Cases	Test Cases in Valid Suites	Mean Tests / Completed Task	Mean Generation Time (s)	Mean Evaluation Time (s)	Mean Total Time (s)	Median Total Time (s)
Hill Climbing	3,160	3,070	9.01	50.12	7.65	57.77	41.00
AVM	3,275	3,171	9.31	46.41	7.68	54.08	35.87
GPT	2,234	1,721	2.62	21.16	6.50	27.66	24.45
Gemini	2,926	1,579	3.43	2.36	5.09	7.45	4.87

Hill Climbing สร้างกรณีทดสอบทั้งหมด 3,160 กรณี โดยมี 3,070 กรณีอยู่ในชุดทดสอบที่ใช้งานได้ ส่วน AVM สร้างทั้งหมด 3,275 กรณี และมี 3,171 กรณีอยู่ในชุดทดสอบที่ใช้งานได้ ขณะที่ GPT สร้างทั้งหมด 2,234 กรณี และมี 1,721 กรณีอยู่ในชุดทดสอบที่ใช้งานได้ ส่วน Gemini สร้างทั้งหมด 2,926 กรณี และมี 1,579 กรณีอยู่ในชุดทดสอบที่ใช้งานได้

เมื่อพิจารณาเฉพาะงานทดลองที่ดำเนินการสำเร็จ Hill Climbing และ AVM สร้างกรณีทดสอบเฉลี่ย 9.01 และ 9.31 กรณีต่องาน ตามลำดับ ขณะที่ GPT สร้างเฉลี่ย 2.62 กรณีต่องาน และ Gemini สร้างเฉลี่ย 3.43 กรณีต่องาน

จำนวนกรณีทดสอบไม่สามารถใช้แทนปริมาณการทดสอบทั้งหมดได้โดยตรง เนื่องจากชุดทดสอบของ AI อาจเรียกเมธอดด้วยข้อมูลนำเข้าหลายค่าและมีคำสั่งตรวจสอบผลหลายรายการภายในกรณีทดสอบเดียว ขณะที่ Hill Climbing และ AVM มักแยก candidate ของชุดทดสอบเป็นหลายกรณีทดสอบ ดังนั้นควรพิจารณาจำนวนกรณีทดสอบร่วมกับจำนวนข้อมูลนำเข้า จำนวนครั้งที่เรียกเมธอด จำนวนคำสั่งตรวจสอบผล ความครอบคลุมของโค้ด และความสามารถในการตรวจพบบั๊ก

ตารางที่ 4.4 ประสิทธิภาพต่อกรณีทดสอบ

Method	Instruction Coverage / Test Case (percentage points)	Branch Coverage / Test Case (percentage points)	Line Coverage / Test Case (percentage points)	Total Time / Test Case (s)
Hill Climbing	4.32	1.47	4.51	9.11
AVM	4.27	1.46	4.46	7.81
GPT	13.95	7.44	14.14	15.15
Gemini	11.33	5.69	11.57	3.17

ผลด้านเวลาแสดงว่า Gemini ใช้เวลาสร้างชุดทดสอบน้อยที่สุดในข้อมูลชุดนี้ ส่วน AVM ใช้เวลาน้อยกว่า Hill Climbing เล็กน้อย อย่างไรก็ตาม เวลาเหล่านี้ไม่รวมการนำซอร์สโค้ดออกจากระบบควบคุมรุ่นและการคอมไพล์โปรแกรมในขั้นเตรียมการ อีกทั้งไม่มีข้อมูลสภาพฮาร์ดแวร์ของผู้ทดลองครบทุกเครื่องในรูปแบบเดียวกัน จึงไม่ควรใช้ผลนี้สรุปความเร็วของอัลกอริทึมหรือโมเดลโดยทั่วไป

4.4 ผลบนเคสเดียวกัน

ใน 349 เคสที่ทุกวิธีดำเนินการสำเร็จ HC, AVM, GPT และ Gemini มีชุดทดสอบที่ใช้งานได้ 344 / 344 / 316 / 254 ชุด และตรวจพบบั๊กได้ 12 / 11 / 20 / 4 เคส ตามลำดับ การเปรียบเทียบนี้ใช้รายการเคสเดียวกัน แต่ค่าเฉลี่ยความครอบคลุมของโค้ดยังคำนวณจากชุดทดสอบที่ใช้งานได้ต่างจำนวนกัน รายละเอียดอยู่ใน summary_common_completed.csv

ใน 231 เคสที่ทุกวิธีดำเนินการสำเร็จและมีชุดทดสอบที่ใช้งานได้ ค่าเฉลี่ยในตารางต่อไปนี้คำนวณจากเคสชุดเดียวกันทั้งหมด

ตารางที่ 4.5 ผลการเปรียบเทียบบนเคสเดียวกัน

Method	Bugs Detected (n, %) / 231 Cases	Instruction Coverage (%)	Branch Coverage (%)	Line Coverage (%)	Mean Total Time (s)
Hill Climbing	8 (3.46%)	21.12%	9.93%	22.55%	53.82
AVM	7 (3.03%)	21.46%	10.25%	22.81%	50.07
GPT	10 (4.33%)	25.11%	14.26%	25.94%	26.82
Gemini	3 (1.30%)	23.53%	12.36%	24.22%	9.75

GPT มีความครอบคลุมของโค้ดเฉลี่ยและจำนวนเคสที่ตรวจพบบั๊กสูงที่สุดในการเปรียบเทียบกลุ่มนี้ แต่จำนวนเคสที่ GPT และ HC ตรวจพบต่างกันเพียง 2 เคส ดังนั้นผลต่าง 38 กับ 12 เคสในตารางผลรวมจึงมีส่วนมาจากความสามารถในการสร้างชุดทดสอบที่ใช้งานได้บนเคสที่อัลกอริทึมเชิงค้นหาไม่รองรับ กลุ่มเคสที่ทุกวิธีมีชุดทดสอบใช้งานได้คัดเลือกเฉพาะเคสที่ทุกวิธีดำเนินการได้ จึงไม่ได้เป็นตัวแทนของทั้งหมด 854 เคส อีกทั้งยังมีสอง experiment_id และไม่มีการทดสอบนัยสำคัญทางสถิติ รายชื่อเคสอยู่ใน cases_common_valid.csv และผลสรุปอยู่ใน summary_common_valid.csv

4.5 การตรวจพบบั๊กแยกตามโปรเจกต์

ตารางที่ 4.6 การตรวจพบบั๊กแยกตามโปรเจกต์

Project	Cases	HC	AVM	GPT	Gemini
Chart	26	2	2	1	0
Cli	39	1	1	3	0
Closure	174	0	0	2	0
Codec	18	1	1	0	0
Collections	28	1	1	0	1
Compress	47	2	2	5	0
Csv	16	1	1	1	0
Gson	18	0	0	0	1
JacksonCore	26	0	0	1	0
JacksonDatabind	110	1	1	2	2
JacksonXml	6	0	0	0	0
Jsoup	93	1	1	3	0
JXPath	22	0	0	1	1
Lang	61	1	1	4	0
Math	106	0	0	8	3
Mockito	38	0	0	5	0
Time	26	1	0	2	0
รวม	854	12	11	38	8

ผลการตรวจพบก็แตกต่างกันระหว่างโปรเจกต์ อัลกอริทึมเชิงค้นหาตรวจพบเคสใน Chart ได้มากกว่า GPT ส่วน GPT ตรวจพบหลายเคสใน Math และ Mockito และ Gemini ตรวจพบเคสบางส่วนที่วิธีอื่นไม่พบ ใน Closure ทั้ง HC และ AVM ดำเนินการสำเร็จเพียง 12 จาก 174 เคส ส่วน GPT และ Gemini มีชุดทดสอบที่ใช้งานได้ 44 / 19 เคส ตามลำดับ ผลนี้สะท้อนความยากของโค้ดต้นฉบับ API และการเตรียมสถานะในระบบที่ใช้ทดลอง มากกว่าการสรุปจากความครอบคลุมของโค้ดเฉลี่ยเพียงอย่างเดียว รายละเอียดอยู่ใน summary_by_project.csv

4.6 ปัญหาที่พบและผลกระทบ

4.6.1 ข้อจำกัดของขอบเขตการค้นหาและการสร้างออบเจกต์

HC และ AVM มีงานทดลองที่ไม่รองรับเท่ากัน คือ 503 เคส โดยจำแนกสาเหตุได้ดังตารางต่อไปนี้

ตารางที่ 4.7 สาเหตุของเคสที่ไม่รองรับ

Unsupported Reason	HC	AVM	Explanation
กลไกวางแผนการสร้างออบเจกต์สร้างออบเจกต์รับการเรียกเมธอดไม่ได้ภายใต้ข้อจำกัดของ public API	380	380	ระบบสร้างออบเจกต์ที่จำเป็นก่อนเริ่มทดสอบไม่ได้
ไม่มีเมธอดสาธารณะที่รับข้อมูลชนิดพื้นฐานหรือ String ตามที่ระบบรองรับ	94	94	ไม่พบเมธอดที่ระบบสร้างข้อมูลนำเข้าเพื่อทดสอบได้
ไม่พบ candidate ที่ใช้งานได้ในออบเจกต์รับการเรียกเมธอด การเตรียมสถานะและเป้าหมายที่เลือก	29	29	ไม่พบชุดทดสอบที่ใช้งานได้ภายในขอบเขตที่กำหนด

ผลการไม่รองรับ 503 เคสต่อวิธีเกิดจากข้อจำกัดของระบบสร้างชุดทดสอบที่ใช้ในการทดลอง ไม่ได้พิสูจน์ว่า Hill Climbing หรือ AVM ใช้กับบิกเหล่านี้ไม่ได้โดยหลักการ สาเหตุที่พบมากที่สุดคือระบบสร้างออบเจกต์ที่รับการเรียกเมธอดไม่ได้ จำนวน 380 เคสต่อวิธี รองลงมาคือไม่มีเมธอดที่อยู่ในขอบเขตชนิดข้อมูลนำเข้าที่รองรับ จำนวน 94 เคสต่อวิธี และไม่พบ candidate ที่ใช้งานได้ จำนวน 29 เคสต่อวิธี

4.6.2 ปัญหาการคอมไพล์และการตรวจสอบชุดทดสอบจาก AI

ตารางที่ 4.8 ผลการคอมไพล์ชุดทดสอบ

Fixed-Version Outcome (Completed Tasks)	HC	AVM	GPT	Gemini
คอมไพล์ไม่ผ่าน	0	0	91	343
คอมไพล์ผ่าน แต่ชุดทดสอบไม่ผ่าน	5	6	197	82

GPT มีงานทดลองที่ดำเนินการสำเร็จแต่ชุดทดสอบใช้งานไม่ได้ 288 งาน แบ่งเป็นคอมไพล์ไม่ผ่าน 91 งาน และคอมไพล์ผ่านแต่รันบนโปรแกรมรุ่นที่แก้ไขแล้วไม่ผ่าน 197 งาน ส่วน Gemini มี 425 งาน แบ่งเป็น 343 และ 82 งานตามลำดับ ผลนี้แสดงว่าการได้รับโค้ดชุดทดสอบภาษา Java จาก AI ยังไม่เพียงพอ ต้องตรวจทั้งการคอมไพล์และการรันบนโปรแกรมรุ่นที่แก้ไขแล้วก่อนนำไปตรวจพบอีก อย่างไรก็ตาม การรันไม่ผ่านอาจเกิดจากคำสั่งตรวจสอบผล การเตรียมสถานะ ข้อยกเว้น หรือปัญหาขณะรัน จึงไม่ควรสรุปว่าทุกงานเกิดจากการคาดเดาผลผลิตของโมเดลโดยไม่ตรวจสอบผลลัพธ์ของแต่ละงานทดลอง

4.6.3 ข้อผิดพลาดที่ยังคงอยู่ในชุดข้อมูล

ตารางที่ 4.9 ข้อผิดพลาดที่ยังคงอยู่ในผลการทดลอง

Error Type	Tasks	Case Evidence
เก็บความครอบคลุมของโค้ดด้วย JaCoCo ในขั้นสุดท้ายไม่สำเร็จ	4	GPT/Cli-16, GPT/Cli-21, GPT/Compress-42, Gemini/Compress-42
การประเมิพบ NoneType ... splitlines	3	HC/Lang-11, HC/Lang-12, AVM/Lang-12
เตรียมหรือส่งออกข้อมูลจาก Defects4J เกินเวลาที่กำหนด	1	GPT/Time-26
เกิดข้อผิดพลาดด้าน DNS หรือเครือข่ายของเกตเวย์	1	GPT/Time-7

ในงานทดลองที่มีข้อผิดพลาด 9 งาน มี 4 งานที่ชุดทดสอบผ่านบนโปรแกรมรุ่นที่แก้ไขแล้ว และบันทึก `valid_test=True` แต่เก็บข้อมูลความครอบคลุมของโค้ดด้วย JaCoCo ไม่สำเร็จ จึงยังมีสถานะเป็นข้อผิดพลาดและไม่นับรวมในผลหลัก เคส GPT/Cli-21 ยังบันทึก `fault_detected=True` ด้วย หากนับเฉพาะค่านี้โดยไม่ตรวจสอบสถานะ จะได้ GPT ตรวจพบอีก 39 เคส แต่เกณฑ์ผลหลักนับเฉพาะงานทดลองที่ดำเนินการสำเร็จและมีชุดทดสอบใช้งานได้ จึงรายงาน 38 เคส

บทที่ 5

สรุปผลการเรียนรู้และอภิปรายผล

5.1 สรุปผล

ต้องพิจารณาแยกกันระหว่างความถูกต้องของชุดทดสอบในกรณีที่สามารถสร้างได้สำเร็จ กับความสามารถของระบบในการรองรับชุดข้อมูลทั้งหมด โดย Hill Climbing และ AVM มีอัตราชุดทดสอบที่ใช้งานได้เกือบร้อยละ 99 เมื่อพิจารณาเฉพาะงานทดลองที่ดำเนินการสำเร็จเนื่องจากผลลัพธ์คาดหวังถูกสร้างจากการรันบน fixed revision อย่างไรก็ตาม เมื่อพิจารณาจากเคสทั้งหมด 854 เคส ทั้งสองวิธีมีส่วนชุดทดสอบที่ใช้งานได้เพียงร้อยละ 40.28 เนื่องจากข้อจำกัดด้านการสร้างออบเจกต์และขอบเขตการค้นหา ดังนั้นการรายงานเพียงเปอร์เซ็นต์เดียวอาจทำให้ตีความประสิทธิภาพของระบบคลาดเคลื่อนได้

ผลการทดลองยังแสดงให้เห็นว่า ความครอบคลุมของโค้ดที่สูงกว่าไม่ได้หมายความว่า จะตรวจพบบั๊กได้มากกว่าเสมอไป โดย AVM มีค่าความครอบคลุมเฉลี่ยสูงกว่า Hill Climbing เล็กน้อย แต่ตรวจพบบั๊กได้น้อยกว่า ขณะที่ Gemini มีค่าความครอบคลุมของชุดทดสอบที่ใช้งานได้ใกล้เคียงกับ GPT แต่มีจำนวนบั๊กที่ตรวจพบน้อยกว่าอย่างชัดเจน สะท้อนให้เห็นว่าการเข้าถึง branch เพียงอย่างเดียวไม่เพียงพอ แต่ยังคงมีข้อมูลนำเข้าและสถานะที่สามารถกระตุ้นข้อบกพร่อง รวมถึง test oracle ที่สามารถสังเกตความแตกต่างของพฤติกรรมได้

ความสามารถในการสร้างออบเจกต์และเตรียมสถานะของโปรแกรมเป็นอีกปัจจัยสำคัญต่อการสร้างชุดทดสอบ เนื่องจากเคสที่ไม่รองรับส่วนใหญ่ของ Hill Climbing และ AVM เกิดขึ้นก่อนที่กระบวนการค้นหาข้อมูลนำเข้าจะเริ่มต้นจริง ดังนั้นการขยายความสามารถของ construction planner และกลไกการเตรียมสถานะอาจช่วยเพิ่มจำนวนเคสที่สามารถทดสอบได้ อย่างไรก็ตาม การทดลองครั้งนี้ยังไม่ได้พิสูจน์โดยตรงว่าการปรับปรุงดังกล่าวจะเพิ่มความสามารถในการตรวจพบบั๊กได้มากนักหรือไม่

สำหรับเครื่องมือ AI พบว่าสามารถสร้างชุดทดสอบได้รวดเร็ว แต่ชุดทดสอบที่ได้อาจจำเป็นต้องผ่านการคอมไพล์และตรวจสอบบน fixed revision ก่อนนำไปใช้ โดย Gemini มีกรณีคอมไพล์ไม่ผ่าน 343 งาน ขณะที่ GPT มีกรณีคอมไพล์ไม่ผ่าน 91 งาน และมีอีก 197 งานที่คอมไพล์ผ่านแต่ชุดทดสอบไม่ผ่านบน fixed revision ผลดังกล่าวแสดงให้เห็นว่ากระบวนการตรวจสอบอัตโนมัติเป็นส่วนสำคัญของการใช้ AI เพื่อสร้างชุดทดสอบ

นอกจากนี้ จำนวน กรณีทดสอบ ไม่สามารถใช้แทนปริมาณการทดสอบทั้งหมดได้โดยตรง เนื่องจากหนึ่งกรณีทดสอบอาจประกอบด้วยข้อมูลนำเข้า การเรียกเมธอด และ assertion หลายรายการ ดังนั้นควรพิจารณาจำนวนการเรียกเมธอด จำนวน assertion คุณภาพของ test oracle และความครอบคลุมของโค้ดร่วมด้วย

แต่ละวิธียังสามารถตรวจพบบั๊กในกลุ่มที่แตกต่างกัน โดยเมื่อรวมผลจากทั้งสี่วิธีสามารถตรวจพบบั๊กที่ไม่ซ้ำกันได้ 51 เคส แสดงให้เห็นถึงประโยชน์ของความหลากหลายของแนวทาง อย่างไรก็ตาม

งานนี้ยังไม่ได้ทดลองระบบผสมที่นำชุดทดสอบจากหลายวิธีมาคัดเลือกร่วมกัน จึงไม่สามารถสรุปประสิทธิภาพของแนวทางผสมได้

สุดท้าย การเก็บเพียง Git commit ยังไม่เพียงพอสำหรับการทำซ้ำการทดลองอย่างสมบูรณ์ ควรเก็บซอร์สโค้ดที่ใช้จริง configuration, prompt, experiment hash, environment, ผลตอบกลับจาก AI และโค้ดชุดทดสอบที่สร้างขึ้นร่วมกัน เพื่อรักษาความสามารถในการตรวจสอบย้อนกลับของผลการทดลอง

5.2 ข้อจำกัด

- 5.2.1 Hill Climbing และ AVM ใช้ค่าตั้งต้นของการสุ่มเพียงค่าเดียว และแต่ละวิธีดำเนินการเพียงครั้งเดียวต่อเคส จึงไม่มีการวัดความแปรปรวนหรือการทดสอบนัยสำคัญจากการทำซ้ำ อีกทั้งไม่มีข้อมูลรุ่นภายในของโมเดลที่ให้บริการผ่านเกตเวย์
- 5.2.2 การเลือกคลาสแบบทราบเป้าหมาย การจำกัดความยาวซอร์สโค้ดของโปรแกรมรุ่นที่แก้ไขแล้ว และการใส่รหัสบักในพรอมต์ ทำให้การทดลองไม่ได้ประเมินการค้นหาคำตอบที่ทราบตำแหน่งล่วงหน้า อีกทั้งยังไม่สามารถตัดโอกาสที่โมเดลจะเคยจดจำข้อมูลที่เกี่ยวข้องได้อย่างสมบูรณ์
- 5.2.3 แต่ละวิธีมีขอบเขตการค้นหา กลไกการสร้างออบเจกต์ จำนวนกรณีทดสอบ งบประมาณ และทรัพยากรที่ใช้ รวมถึงพรอมต์แตกต่างกัน ผลลัพธ์จึงอธิบายได้เฉพาะระบบที่ใช้ในการทดลองครั้งนี้
- 5.2.4 ค่าเฉลี่ยความครอบคลุมของโค้ดและเวลาในผลรวมคำนวณจากกลุ่มงานทดลองที่ผ่านเกณฑ์ต่างกัน การใช้กลุ่มเคสที่ทุกวิธีมีชุดทดสอบใช้งานได้ช่วยให้เปรียบเทียบบนรายการเคสเดียวกัน แต่ยังคงมีความเอนเอียงจากการคัดเลือก และไม่ได้จัดข้อจำกัดด้านสภาพแวดล้อมและการกำหนดค่า
- 5.2.5 ข้อมูลสภาพฮาร์ดแวร์และรุ่นซอฟต์แวร์ของผู้ทดลองแต่ละคนยังไม่ครบถ้วน อีกทั้งไม่มีข้อมูลราคา จึงไม่สามารถสรุปความเร็วบนเครื่องเดียวกันหรือความคุ้มค่าด้านราคาได้
- 5.2.6 กลไกตรวจพบบักใช้การล้มเหลวของกระบวนการรันโปรแกรมรุ่นที่ยังมีข้อบกพร่องเป็นเกณฑ์ การยืนยันชนิดของข้อบกพร่องในแต่ละเคสจึงต้องตรวจรายละเอียดการทดสอบที่ไม่ผ่านจาก JUnit เพิ่มเติม
- 5.2.7 ข้อมูลกำกับบางส่วน ข้อความช่วงท้ายของผลการรัน และสัดส่วนความครอบคลุมของโค้ดที่เก็บไว้เพียงพอสำหรับสรุปผลซ้ำจากข้อมูลเดิม แต่ไม่สามารถทดแทนบันทึกการรันและข้อมูลความครอบคลุมของโค้ดฉบับเต็มที่ไม่ได้บันทึกไว้

5.3 ข้อสรุปและแนวทางพัฒนา

ภายใต้ระบบและข้อมูลที่ใช้ในการทดลองครั้งนี้ GPT มีสัดส่วนชุดทดสอบที่ใช้งานได้จากเคสทั้งหมดและจำนวนเคสที่ตรวจพบบักสูงสุด ส่วน Gemini ใช้เวลาเฉลี่ยต่องานทดลองน้อยที่สุด HC และ AVM มีอัตราชุดทดสอบที่ใช้งานได้สูงเมื่อระบบรองรับเคส โดย AVM มีความครอบคลุมของโค้ดสูงกว่า HC เล็กน้อยและใช้เวลาน้อยกว่า แต่ HC ตรวจพบบักได้เพิ่ม 1 เคส เมื่อเทียบกับทั้งหมด 854 เคส ทุกวิธียังมีอัตราการตรวจพบบักต่ำ จึงควรพิจารณาผลลัพธ์คาดหวัง (test oracle) การเตรียมสถานะ และการเลือกคลาสเป้าหมายร่วมกับความครอบคลุมของโค้ด ผลนี้ไม่เพียงพอที่จะสรุปว่าวิธีใดดีที่สุดโดยทั่วไป

การทดลองครั้งต่อไปควรกำหนดชุดการทดลองเดี่ยวและตรึงซอร์สโค้ด พรอมต์ และการกำหนดค่าไว้ พร้อมเก็บข้อมูลสภาพแวดล้อมและฮาร์ดแวร์ ใช้ค่าตั้งต้นของการสุ่มหลายค่าและรันซ้ำหลายครั้ง กำหนดงบประมาณทรัพยากรและนโยบายจำนวนกรณีทดสอบร่วมกัน ตรวจคลาสเป้าหมาย API และการคอมไพล์ชุดทดสอบจาก AI พร้อมขั้นตอนแก้ไขโค้ดที่วัดต้นทุนแยก และบันทึกผลจาก JUnit กับความครอบคลุมของโค้ดฉบับเต็ม การขยายกลไกวางแผนการสร้างออบเจกต์หรือใช้หลายวิธีร่วมกันเป็นข้อเสนอที่ต้องทดลองเพิ่มเติม และยังไม่ได้รับการพิสูจน์ในการทดลองครั้งนี้

เอกสารอ้างอิง

- [1] Just, R., Jalali, D., และ Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. Proceedings of the International Symposium on Software Testing and Analysis (ISSTA).
<https://doi.org/10.1145/2610384.2628055>
- [2] Mountainminds GmbH & Co. KG and Contributors. (ม.ป.ป.). JaCoCo Coverage Counters. สืบค้น เมื่อ 1 ตุลาคม 2569.
<https://www.jacoco.org/jacoco/trunk/doc/counters.html>
- [3] JUnit Team. (ม.ป.ป.). JUnit 4. สืบค้น เมื่อ 1 ตุลาคม 2569. <https://junit.org/junit4/>
- [4] McMin, P. (2004). Search-based software test data generation: A survey. Software Testing, Verification and Reliability, 14(2), 105–156.
<https://doi.org/10.1002/stvr.294>
- [5] McMin, P., และ Kapfhammer, G. M. (2016). AVMf: An open-source framework and implementation of the Alternating Variable Method. Search-Based Software Engineering, Lecture Notes in Computer Science, 9962, 259–266.
https://doi.org/10.1007/978-3-319-47106-8_21